POLARS VS PANDAS

1. Objective

The objective of this assignment is to compare Pandas and Polars for data manipulation and analysis. The comparison covers execution time, memory use, scalability, usability, and correctness for common data-processing operations.

2. Introduction

Pandas is a popular Python library for working with tabular data. It provides DataFrame and Series structures and is widely used for data cleaning, analysis, and exploratory data analysis.

Polars is a DataFrame library designed for fast data processing. It supports parallel execution and offers both eager execution (perform operations immediately) and lazy execution (build a query plan and optimize it before execution).

Both libraries can load, filter, select, group, join, sort, and transform data. Pandas has a large ecosystem and is familiar to many data analysts. Polars can be especially efficient for large datasets and multi-step processing pipelines.

3. Environment Setup and Tools Used

The benchmark notebook was run in a Python environment. The recorded environment was Python 3.13.16, Pandas 2.2.3, Polars 1.35.2, NumPy 2.1.3, using a 2-core CPU.

Libraries and tools used:

- Pandas — DataFrame operations and reference implementation.
- Polars — eager and lazy DataFrame operations.
- NumPy — generation of reproducible synthetic data and numerical comparisons.
- Matplotlib — charts for execution time, pipeline performance, memory and scalability.
- PyArrow — supporting columnar-data functionality.
- psutil and Python resource — process-memory measurements.
- time.perf_counter, statistics.median and gc — timing repeated runs and reporting median values.
- Jupyter Notebook / Google Colab — running the code cells and viewing outputs.

Installation command used in the notebook:

---

4. Dataset Selection

A synthetic retail dataset was generated using NumPy with a fixed random seed (42). It consists of two CSV files: orders.csv and customers.csv. The data is synthetic and was created for benchmarking; it is not taken from an external website.

Dataset	Rows	Columns	File size (recorded)
orders.csv	5,000,000	11	341 MB

customers.csv	200,000	5	6.1 MB

Orders fields include order_id, customer_id, product_id, category, region, quantity, unit_price, discount, order_date, payment_method and is_returned. The customers table contains customer_id and customer attributes such as segment, country, age and other customer information. The common key used to join the two tables is customer_id.

5. Implementation and Operations

- **Load CSV:** Read orders.csv and customers.csv into Pandas and Polars DataFrames. The date column is parsed as a date.
- **Filter rows:** Keep orders in the Electronics category with quantity >= 5, unit_price > 50, and is_returned = False.
- **Select columns:** Select order_id, customer_id, category, quantity and unit_price.
- **Group and aggregate:** Group by category and region; calculate total quantity, average price, number of orders and maximum discount.
- **Join datasets:** Perform an inner join between orders and customers using customer_id.
- **Sort data:** Sort by unit_price in descending order and order_id in ascending order.
- **Transform data:** Create revenue = quantity × unit_price × (1 − discount), extract order year and month, and assign a low/mid/high price tier.
- **End-to-end pipeline:** Filter orders, join customer data, calculate revenue, group by country and category, and sort the aggregated result. This is tested with Pandas, Polars eager and Polars lazy.

6. Correctness / Equivalent Results

Before comparing execution times, the notebook checked that both libraries returned equivalent results. The recorded checks passed for input shapes, filtering, selected columns, group-by aggregation, join, sorting, transformations and the final pipeline.

The filter returned 178,950 rows. Group-by produced 40 category-region groups. The join returned 5,000,000 rows and 15 columns. The transformed orders table contained 5,000,000 rows and 15 columns. The final pipeline result contained 48 rows and 4 columns.

7. Performance Benchmarking Method

Each operation was benchmarked repeatedly. The notebook used time.perf_counter() to measure elapsed time, gc.collect() before each run, and the median of five runs as the main result. Filter, group-by, join and sort were rechecked over 15 runs because their timings varied. The full end-to-end pipeline was measured over three runs. Times are specific to the recorded environment and can change on another computer.

Operation	Pandas median (s)	Polars median (s)	Interpretation
Load both CSV files	9.471	2.538	Polars ~3.73× faster
Filter	0.932	0.062	Polars ~14.91× faster
Select columns	0.237	0.001	Polars faster; very small time, ratio is sensitive to measurement noise
Group-by	1.172	1.101	Close; Polars slightly faster in initial run
Join	1.937	2.114	Pandas slightly faster in initial run
Sort	6.987	3.201	Polars ~2.18× faster
Transform	1.886	0.260	Polars ~7.24× faster

The 15-run re-check gave these medians: filter — Pandas 0.524 s and Polars 0.108 s; group-by — Pandas 1.212 s and Polars 0.756 s; join — Pandas 2.113 s and Polars 2.263 s; sort — Pandas 7.896 s and Polars 3.468 s. The

differences between the first run set and the re-check show that timings can be noisy, particularly on a 2-core environment.

8. End-to-End Pipeline Results

Implementation	Median time
Pandas	15.04 seconds
Polars eager	5.76 seconds
Polars lazy	2.98 seconds

The Polars lazy pipeline was approximately 5.0 times faster than the Pandas pipeline in this recorded run. Lazy execution can optimize a sequence of operations before processing the data.

9. Memory Analysis

The notebook measured table size and process memory. The recorded orders-table size was 1,053 MiB in Pandas and 329 MiB in Polars eager. Based on these table-size measurements, the Polars representation used about 69% less memory for the orders table.

The notebook also reported process-memory values, but its peak-RSS readings were all 6,161 MiB, including the baseline. These peak figures do not distinguish the libraries reliably and should not be used to claim a peak-memory winner. The table-size comparison is the clearer result in this run. Memory figures depend on data types, string storage, runtime and environment.

10. Scalability Testing

The benchmark repeated selected operations on the first 100,000, 500,000, 1,000,000, 2,500,000 and 5,000,000 order rows, using the same 200,000-row customer table. The following values are Polars speedup ratios (Pandas time divided by Polars time). A value above 1 means Polars was faster; below 1 means Pandas was faster.

Operation	100k	500k	1M	2.5M	5M
Filter	6.45×	6.94×	7.34×	8.03×	5.04×
Group-by	3.34×	1.71×	1.63×	2.25×	1.73×
Join	1.27×	0.74×	0.85×	0.70×	0.93×
Sort	2.66×	0.81×	1.74×	1.73×	1.57×
Transform	4.91×	4.51×	6.63×	6.59×	8.45×

Scalability results are operation-dependent. Polars was consistently faster for filtering and transformations in this experiment. Join performance was close and Pandas was faster at several sizes. Sorting results also varied by dataset size, so the results do not support a claim that one library wins every operation.

11. Results and Observations

- Polars loaded the two CSV files faster in the recorded environment.
- Polars performed especially well for filtering, selecting columns and transforming data.
- Group-by performance was relatively close; repeated measurements showed timing variation.
- Pandas was slightly faster for the full join in the initial benchmark and the 15-run re-check.
- Polars sorted the 5-million-row dataset faster in both the initial result and the re-check.
- Polars lazy execution had the best end-to-end pipeline time in this experiment.
- The orders-table memory estimate was smaller in Polars, although process peak-memory readings were inconclusive.
- Correctness checks passed, so the performance comparison was made on equivalent results.

12. Strengths, Limitations and Use Cases

Library	Strengths	Suitable use cases
Pandas	Widely used; readable API; broad ecosystem; good integration with Python data-science tools.	Exploratory data analysis, small-to-medium datasets, notebooks, workflows using Pandas-compatible libraries.
Polars	Fast parallel processing; efficient columnar memory representation; lazy query optimization.	Large tabular datasets, repeated filtering and transformation, multi-step data pipelines.

Limitations: results depend on hardware, data types, dataset shape and code style. The benchmark uses synthetic data, so it may not represent every real-world dataset. Some operations, especially joins, did not show a consistent Polars advantage. Absolute execution times should not be generalized to all systems.

13. Conclusion

This assignment compared Pandas and Polars using a synthetic retail dataset containing five million orders and 200,000 customers. Both libraries completed the required data-processing operations and passed the notebook's equivalence checks. In the recorded benchmark, Polars was faster at CSV loading, filtering, sorting and transformations, and its lazy pipeline completed in 2.98 seconds compared with 15.04 seconds for Pandas. The orders table also had a smaller estimated in-memory size in Polars. However, Pandas was slightly faster for joining the datasets, and group-by performance was relatively close. Therefore, Polars is a strong choice for large, multi-step data-processing workloads, while Pandas remains useful for general data analysis and ecosystem compatibility. The best library should be selected based on the actual workload and tested environment.

14. Files / Outputs Used

The implementation and outputs were taken from the submitted Jupyter Notebook (CODE.ipynb). The notebook includes installation/imports, synthetic dataset creation, matching Pandas and Polars functions, equivalence checks, repeated timing, end-to-end pipeline benchmarking, memory measurements, scalability testing, chart generation and a summary table. Submit the notebook together with this report if the instructor requires both code and report.